

PART 4: Exception Handling & Robust Code Design

4.1 The Throwable Hierarchy

Definition & Internal Mechanics

The `Throwable` class is the root of the Java exception hierarchy. It has two direct subclasses:

`Error`:

- Represents serious JVM-level failures that application code typically cannot recover from
- `OutOfMemoryError`: Heap is full; GC cannot free enough space
- `StackOverflowError`: Thread's call stack depth exceeded (unbounded recursion)
- `VirtualMachineError`: JVM encountered a fatal internal error
- `AssertionError`: An `assert` statement evaluated to `false`
- Best practice: Do NOT catch `Error` classes unless you have a very specific recovery strategy (e.g., catching `OutOfMemoryError` to release a large cache and retry)

`Exception`:

- Represents conditions that application code should handle
- Splits into **Checked** and **Unchecked** exceptions

```
Throwable
├── Error
│   ├── OutOfMemoryError
│   ├── StackOverflowError
│   └── AssertionError
└── Exception
    ├── IOException (checked)
    ├── SQLException (checked)
    ├── RuntimeException (unchecked)
    │   ├── NullPointerException
    │   ├── IllegalArgumentException
    │   ├── IllegalStateException
    │   ├── ClassCastException
    │   └── IndexOutOfBoundsException
    └── (Other checked exceptions)
```

Real-Life Analogy

`Error` is like a building's foundation collapsing — there's nothing an individual tenant can do about it; the building itself is compromised. `Exception` is like a locked door (checked — you planned for it and have a key) or tripping over your own feet (unchecked — this shouldn't happen if you walk carefully).

Interview Context & Why It Matters

Interviewers want to see that you don't blindly catch `Throwable` or `Exception` in production — swallowing errors or catching `OutOfMemoryError` and continuing can corrupt application state.

Production-Ready Code Snippet

```
java
```

```

/**
 * Demonstrates the Throwable hierarchy with proper handling patterns.
 */
public class ThrowableHierarchyDemo {

    // Demonstrating when it makes sense to catch Error (rare, specific cases)
    static void processWithOOMGuard(byte[][] largeCache) {
        try {
            // High-risk allocation
            byte[] massive = new byte[Integer.MAX_VALUE];
        } catch (OutOfMemoryError oom) {
            // Release the large cache to free memory, then log and rethrow
            System.out.println("OOM detected - releasing cache to attempt recovery")
            for (int i = 0; i < largeCache.length; i++) {
                largeCache[i] = null;
            }
            // After freeing memory, rethrow so the JVM/container knows the situation
            throw oom; // Do NOT swallow OOM!
        }
    }

    // Recursive method that causes StackOverflowError
    static int infiniteRecursion(int n) {
        return infiniteRecursion(n + 1); // No base case
    }

    public static void main(String[] args) {
        // StackOverflowError
        try {
            infiniteRecursion(0);
        } catch (StackOverflowError e) {
            System.out.println("Caught StackOverflowError - do NOT continue normally")
            // In production: log this as FATAL, don't retry the same recursive operation
        }

        // Demonstrating catching Exception is too broad - RuntimeExceptions slip through
        // This is an anti-pattern but shown to explain the pitfall:
        try {
            String s = null;
            s.length(); // NullPointerException
        } catch (Exception e) {
            // ANTI-PATTERN: Caught NullPointerException silently!
            // Should have checked for null beforehand, not caught the symptom
        }
    }
}

```

```

        System.out.println("Caught broadly: " + e.getClass().getSimpleName());
    }

    // Best practice: catch specific exceptions
    try {
        int[] arr = new int[5];
        arr[10] = 1; // ArrayIndexOutOfBoundsException
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Specific catch: bounds check failed - " + e.getMessage());
    }
}
}

```

Common Interview Question & Expert Answer

Q: "When, if ever, is it appropriate to catch `Error` or `Throwable`?"

A: "Catching `Throwable` or `Error` is almost always a code smell, but there are legitimate exceptions. The most justified case is in **top-level framework code** like a servlet container, application server, or thread pool executor — code that must guarantee a response is returned to the client even if a single request fails catastrophically. In that context, catching `Throwable` prevents one thread's crash from taking down the whole server. However, the correct pattern is to: (1) **log the `Throwable` immediately with full context**, (2) **return a 500 error response or equivalent**, (3) **NOT continue using the same thread state** — treat the thread as potentially corrupted. `OutOfMemoryError` is a specific case where catching it might enable a last-ditch cache eviction strategy, but you must rethrow it after or terminate the JVM. **Never** catch and swallow `Error` silently — this masks JVM instability and leads to data corruption. In production systems, catching `Throwable` in a framework's request handler is legitimate; in application business logic, it's almost always wrong."

4.2 Checked vs. Unchecked Exceptions

Definition & Internal Mechanics

Checked Exceptions:

- Subclasses of `Exception` (but NOT `RuntimeException`)
- The compiler enforces handling: either `try-catch` or declare with `throws`
- Designed for **expected, recoverable conditions** the caller should handle
- Examples: `IOException`, `SQLException`, `ClassNotFoundException`, `InterruptedException`

Unchecked Exceptions (RuntimeException):

- Subclasses of `RuntimeException` or `Error`
- No compiler enforcement — can propagate silently up the call stack
- Designed for **programming errors** or conditions the caller can't meaningfully recover from
- Examples: `NullPointerException`, `IllegalArgumentException`, `ArrayIndexOutOfBoundsException`

The Controversy: Checked exceptions have become controversial in modern Java design:

- **For checked:** Forces callers to acknowledge failure modes, documents failure conditions in the API
- **Against checked:** Leads to `catch (Exception e) {}` boilerplate, exception chaining overhead, difficult to use with lambdas and streams (functional interfaces don't declare checked exceptions)

Modern trend: Many frameworks (Spring, Hibernate) wrap checked exceptions in `RuntimeException` subclasses (e.g., Spring's `DataAccessException`) to avoid the propagation burden.

Real-Life Analogy

Checked exceptions are like a pilot's required checklist — "you MUST verify fuel before takeoff; the cockpit won't let you skip it." **Unchecked exceptions** are like unexpected turbulence — the pilot doesn't pre-plan for every possible turbulence scenario, but if something unexpected happens, there are emergency protocols.

Interview Context & Why It Matters

Interviewers test whether you understand the design intent — checked exceptions for expected failures the caller should handle, unchecked for programmer errors. Using checked exceptions for every failure leads to unreadable code; using unchecked everywhere hides failure modes from API consumers.

Production-Ready Code Snippet

```
java
```

```

import java.io.*;
import java.nio.file.*;
import java.util.Optional;

/**
 * Demonstrates checked vs unchecked exceptions, proper exception chaining,
 * and the modern pattern of wrapping checked in domain-specific unchecked exception
 */
public class ExceptionTypesDemo {

    // Custom checked exception – use when callers MUST handle this case
    static class ConfigurationException extends Exception {
        public ConfigurationException(String message) { super(message); }
        public ConfigurationException(String message, Throwable cause) { super(message, cause); }
    }

    // Custom unchecked exception – use for programming errors or unrecoverable failures
    static class InvalidOrderStateException extends RuntimeException {
        private final String orderId;

        public InvalidOrderStateException(String orderId, String message) {
            super("Order " + orderId + ": " + message);
            this.orderId = orderId;
        }

        public String getOrderId() { return orderId; }
    }

    // Method declaring a checked exception – forces callers to handle it
    static String loadConfig(String filePath) throws ConfigurationException {
        try {
            return Files.readString(Path.of(filePath));
        } catch (IOException e) {
            // Exception chaining: preserve original cause for diagnostics
            throw new ConfigurationException("Failed to load config from: " + filePath, e);
        }
    }

    // Spring-style: wraps checked in unchecked for stream/lambda compatibility
    @FunctionalInterface
    interface CheckedSupplier<T> {
        T get() throws Exception;
    }
}

```

```

static <T> T unchecked(CheckedSupplier<T> supplier) {
    try {
        return supplier.get();
    } catch (RuntimeException e) {
        throw e; // rethrow unchecked as-is
    } catch (Exception e) {
        throw new RuntimeException(e); // wrap checked in unchecked
    }
}

static void validateOrderTransition(String orderId, String currentStatus, String
    if (currentStatus.equals("DELIVERED") || currentStatus.equals("CANCELLED"))
        throw new InvalidOrderStateException(orderId,
            "Cannot transition from terminal state: " + currentStatus + " → " +
    }
}

public static void main(String[] args) {
    // Checked exception handling – MUST handle or declare
    try {
        String config = loadConfig("/nonexistent/config.properties");
    } catch (ConfigurationException e) {
        System.out.println("Config error: " + e.getMessage());
        System.out.println("Caused by: " + e.getCause().getClass().getSimpleName()
    }

    // Unchecked exception – can propagate silently
    try {
        validateOrderTransition("ORD-001", "DELIVERED", "PROCESSING");
    } catch (InvalidOrderStateException e) {
        System.out.println("Caught business exception for order: " + e.getOrderId()
        System.out.println("Message: " + e.getMessage());
    }

    // Using Optional to avoid NullPointerException (alternative to checked exce
    Optional<String> configValue = Optional.ofNullable(System.getProperty("nonex
        .map(String::trim)
        .filter(s -> !s.isEmpty());
    String value = configValue.orElse("default-value");
    System.out.println("Config value: " + value);

    // Wrapping checked for use in streams (common pattern)
    java.util.List<String> files = java.util.List.of("/tmp/a.txt", "/tmp/b.txt")

```

```
// Without helper: can't use Files.readString in a lambda (it throws IOException)
// With helper:
files.stream()
    .map(f -> unchecked(() -> "file: " + f)) // Checked wrapped in unchecked
    .forEach(System.out::println);
}
}
```

Common Interview Question & Expert Answer

Q: "Why can't you use a method that throws a checked exception directly inside a Java stream's `map()` or `filter()` lambda? How do you work around it?"

A: "The `Function<T, R>` functional interface used by `Stream.map()` has the signature `R apply(T t)` — it doesn't declare any checked exceptions. So the lambda must also not throw checked exceptions, because the interface contract doesn't allow it. If you try to write `stream.map(f -> Files.readString(Path.of(f)))`, the compiler rejects it because `Files.readString()` throws `IOException` (checked) but `Function.apply()` doesn't declare `throws IOException`. There are three common workarounds: (1) **Wrap in try-catch inside the lambda** — converts the checked exception to an unchecked `RuntimeException` and rethrows. (2) **Create a functional wrapper** like `CheckedFunction<T, R>` that catches and wraps the checked exception. (3) **Use a library helper** like Guava's `Throwables.propagate()` or Lombok's `@SneakyThrows`. The cleanest production pattern is creating a wrapper method that converts the checked exception: `private String readFileUnchecked(String path) { try { return Files.readString(...); } catch (IOException e) { throw new UncheckedIOException(e); } }` — then use `stream.map(this::readFileUnchecked)`. Note: `UncheckedIOException` is a standard JDK class specifically for wrapping `IOException`."

4.3 Custom Business Exceptions

Definition & Internal Mechanics

Custom exceptions allow you to create a **domain-specific exception hierarchy** that maps precisely to your business failures. Benefits:

- Callers can catch specific exceptions without catching broad superclasses
- Exception message and additional fields carry domain context
- `catch` blocks can have separate handling logic per failure type
- Exceptions self-document failure modes better than generic errors

Design Guidelines:

- Extend `RuntimeException` for most domain exceptions in modern Java (less boilerplate)
- Use checked exceptions only for failures that callers **MUST** handle (e.g., configuration loading)
- Always include both `String message` and `Throwable cause` constructors for exception chaining
- Add domain-specific fields (`orderId`, `accountId`, HTTP status code, etc.) for richer diagnostics
- Create a hierarchy: `AppException` → `PaymentException` → `InsufficientFundsException`

Real-Life Analogy

Custom exceptions are like specialized error codes on medical equipment. Instead of every malfunction displaying "Error 1," you get "Error 105: Probe temperature too high — disconnect patient immediately." The specific error code tells the doctor *exactly* what went wrong and what to do next — no guessing required.

Production-Ready Code Snippet

```
java
```

```

import java.util.Map;

/**
 * Demonstrates a production-quality custom exception hierarchy for a payment system
 * Includes error codes, domain context, HTTP status mapping, and proper chaining.
 */
public class CustomExceptionsDemo {

    // Base exception for all application errors
    static class AppException extends RuntimeException {
        private final String errorCode;

        protected AppException(String errorCode, String message) {
            super(message);
            this.errorCode = errorCode;
        }

        protected AppException(String errorCode, String message, Throwable cause) {
            super(message, cause);
            this.errorCode = errorCode;
        }

        public String getErrorCode() { return errorCode; }
    }

    // Domain-specific: payment failures
    static class PaymentException extends AppException {
        private final String paymentId;

        public PaymentException(String errorCode, String paymentId, String message) {
            super(errorCode, message);
            this.paymentId = paymentId;
        }

        public PaymentException(String errorCode, String paymentId, String message,
            super(errorCode, message, cause);
            this.paymentId = paymentId;
        }

        public String getPaymentId() { return paymentId; }
    }

    // Specific: insufficient funds – callers might retry with a different payment m

```

```

static class InsufficientFundsException extends PaymentException {
    private final double requested;
    private final double available;

    public InsufficientFundsException(String paymentId, double requested, double
        super("PAY-001", paymentId,
            String.format("Insufficient funds: requested=%.2f, available=%.2f",
                this.requested = requested;
                this.available = available;
    }

    public double getRequested() { return requested; }
    public double getAvailable() { return available; }
    public double getShortfall() { return requested - available; }
}

// Specific: account locked – caller must unlock before retrying
static class AccountLockedException extends PaymentException {
    private final String accountId;

    public AccountLockedException(String paymentId, String accountId) {
        super("PAY-002", paymentId, "Account " + accountId + " is locked");
        this.accountId = accountId;
    }

    public String getAccountId() { return accountId; }
}

// Payment service using the hierarchy
static class PaymentService {
    private static final Map<String, Double> BALANCES = Map.of(
        "ACC-001", 500.0,
        "ACC-002", 50.0
    );
    private static final java.util.Set<String> LOCKED_ACCOUNTS =
        java.util.Set.of("ACC-003");

    public void processPayment(String paymentId, String accountId, double amount) {
        if (LOCKED_ACCOUNTS.contains(accountId)) {
            throw new AccountLockedException(paymentId, accountId);
        }

        Double balance = BALANCES.get(accountId);
        if (balance == null) {

```

```

        throw new ApplicationException("ACC-001", "Account not found: " + accountId);
    }

    if (balance < amount) {
        throw new InsufficientFundsException(paymentId, amount, balance);
    }

    System.out.println("Payment " + paymentId + " processed: $" + amount + "
}
}

public static void main(String[] args) {
    PaymentService service = new PaymentService();
    String[][] tests = {
        {"PAY-001", "ACC-001", "200.0"}, // Success
        {"PAY-002", "ACC-002", "100.0"}, // InsufficientFunds
        {"PAY-003", "ACC-003", "50.0"}, // AccountLocked
        {"PAY-004", "ACC-999", "50.0"}, // AccountNotFound
    };

    for (String[] test : tests) {
        String paymentId = test[0], accountId = test[1];
        double amount = Double.parseDouble(test[2]);
        try {
            service.processPayment(paymentId, accountId, amount);
        } catch (InsufficientFundsException e) {
            System.out.printf("[%s] Insufficient funds: need $%.2f more. ErrorCo
                e.getPaymentId(), e.getShortfall(), e.getErrorCode());
        } catch (AccountLockedException e) {
            System.out.printf("[%s] Account %s locked. ErrorCode: %s%n",
                e.getPaymentId(), e.getAccountId(), e.getErrorCode());
        } catch (AppException e) {
            System.out.printf("Application error [%s]: %s%n", e.getErrorCode(),
        }
    }
}
}

```

Common Interview Question & Expert Answer

Q: "Should custom business exceptions extend `Exception` (checked) or `RuntimeException` (unchecked)? What's the tradeoff?"

A: "The industry has largely moved toward `RuntimeException` for business exceptions, for several reasons: First, checked exceptions are **incompatible with functional interfaces** — you can't propagate them through `Stream.map()`, `CompletableFuture.thenApply()`, or most reactive APIs without wrapping them, which creates boilerplate. Second, the 'checked exception forces handling' argument breaks down in practice — developers write `catch(Exception e) {}` to silence the compiler rather than meaningfully handling the error, making checked exceptions counterproductive. Third, Spring, Hibernate, and most major frameworks wrap infrastructure exceptions in unchecked subclasses for exactly this reason. The right approach is: use `RuntimeException` for business domain exceptions, but document them thoroughly in Javadoc with `@throws` so callers know what can fail. Use checked exceptions only where the failure is genuinely expected and recoverable by the immediate caller — for example, a file loading API where the caller needs to fall back to a default if the file doesn't exist. Create a consistent hierarchy: one base (`AppException`) → service-specific (`PaymentException`) → specific causes (`InsufficientFundsException`). This lets callers catch at the right granularity."

4.4 Try-With-Resources and AutoCloseable

Definition & Internal Mechanics

try-with-resources (Java 7+) automatically closes resources declared in the `try` clause when the block exits, whether normally or via exception. The resource must implement `AutoCloseable` (or its sub-interface `Closeable` for I/O).

Execution order:

1. Resource(s) are initialized (left to right)
2. `try` block executes
3. Resources are closed **in reverse order of declaration**
4. If an exception is thrown in `close()` AND an exception was already thrown in the `try` block, the close exception is **suppressed** (attached to the original exception via `addSuppressed()`)

Multiple resources:

```
java
try (InputStream is = new FileInputStream("a");
    InputStreamReader isr = new InputStreamReader(is)) { ... }
// isr.close() called first, then is.close()
```

Suppressed exceptions: Accessible via `Throwable.getSuppressed()`. Before Java 7, the common pattern of `finally { resource.close(); }` meant the close exception **replaced** the

original — completely hiding the root cause.

Real-Life Analogy

`try-with-resources` is like a restaurant reservation system with automatic table cleanup. When you (the block) leave the restaurant (exit try), the busboys (resource closers) automatically clear every table you used, starting with the last one, even if you had an altercation (exception) on the way out. The incident report (exception) includes a "minor note" (suppressed exception) if the busboy spilled something while cleaning up.

Interview Context & Why It Matters

Resource leaks (unclosed streams, database connections, sockets) are a leading cause of memory leaks and connection pool exhaustion in production systems. Interviewers test whether you understand the suppression mechanism and the correct resource ordering.

Production-Ready Code Snippet

```
java
```

```

import java.io.*;
import java.nio.file.*;
import java.sql.*;

/**
 * Demonstrates try-with-resources, multiple resources, AutoCloseable,
 * suppressed exceptions, and the pre-Java-7 resource leak problem.
 */
public class TryWithResourcesDemo {

    // Custom AutoCloseable – simulates a database connection
    static class DatabaseConnection implements AutoCloseable {
        private final String url;
        private boolean closed = false;

        public DatabaseConnection(String url) {
            this.url = url;
            System.out.println(" [OPEN] Connection to: " + url);
        }

        public String query(String sql) {
            if (closed) throw new IllegalStateException("Connection is already closed");
            if (sql.contains("BAD")) throw new RuntimeException("Query failed: " + sql);
            return "ResultSet for: " + sql;
        }

        @Override
        public void close() {
            if (!closed) {
                closed = true;
                System.out.println(" [CLOSE] Connection to: " + url);
                // Simulate a close failure (rare but possible):
                // throw new RuntimeException("Failed to close connection");
            }
        }
    }

    static class FileProcessor implements AutoCloseable {
        private final String filename;

        public FileProcessor(String filename) {
            System.out.println(" [OPEN] FileProcessor: " + filename);
            this.filename = filename;
        }
    }
}

```

```

    }

    public void process() {
        System.out.println("    Processing: " + filename);
    }

    @Override
    public void close() {
        System.out.println("    [CLOSE] FileProcessor: " + filename);
    }
}

public static void main(String[] args) {
    System.out.println("=== Single Resource ===");
    try (DatabaseConnection conn = new DatabaseConnection("jdbc:postgresql://localhost:5432/testdb")) {
        System.out.println(conn.query("SELECT * FROM users"));
    } // conn.close() called automatically here

    System.out.println("\n=== Multiple Resources (reverse close order) ===");
    try (DatabaseConnection conn = new DatabaseConnection("jdbc:postgresql://localhost:5432/testdb");
        FileProcessor fp = new FileProcessor("report.csv")) {
        conn.query("SELECT * FROM orders");
        fp.process();
    }
    // fp.close() first, then conn.close() - reverse of declaration order

    System.out.println("\n=== Exception in try block ===");
    try (DatabaseConnection conn = new DatabaseConnection("jdbc:postgresql://localhost:5432/testdb")) {
        conn.query("BAD SQL"); // Throws RuntimeException
    } catch (RuntimeException e) {
        System.out.println("    Caught: " + e.getMessage());
        System.out.println("    Suppressed exceptions: " + e.getSuppressed().length);
    }

    System.out.println("\n=== Suppressed Exception Demo ===");
    // Custom resource that throws on close
    AutoCloseable failingResource = () -> {
        throw new RuntimeException("CLOSE FAILED");
    };

    try (failingResource) {
        throw new RuntimeException("ORIGINAL ERROR");
    } catch (RuntimeException e) {
        System.out.println("Original: " + e.getMessage());
    }
}

```

```

        for (Throwable suppressed : e.getSuppressed()) {
            System.out.println("Suppressed: " + suppressed.getMessage()); // CLO
        }
    }

    System.out.println("\n=== Real File I/O with try-with-resources ===");
    try {
        Path tempFile = Files.createTempFile("demo", ".txt");
        Files.writeString(tempFile, "Hello, Java!\nSecond line.");

        try (BufferedReader reader = Files.newBufferedReader(tempFile)) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println("  Read: " + line);
            }
        } // BufferedReader automatically closed – no finally block needed
        Files.delete(tempFile);
    } catch (IOException e) {
        System.err.println("I/O Error: " + e.getMessage());
    }
}
}

```

Common Interview Question & Expert Answer

Q: "What are 'suppressed exceptions' in Java and why were they introduced with `try-with-resources`?"

A: "Before Java 7, the standard pattern for resource cleanup was `finally { resource.close(); }`. The problem was that if the `try` block threw exception A and then `close()` in `finally` threw exception B, exception B would **replace** exception A completely — the original root cause was swallowed and lost. Developers would see a spurious `NullPointerException` from `close()` while the actual bug was a `SQLException` that caused the connection to be in a bad state. Java 7's `try-with-resources` solved this with **suppressed exceptions**: if the `try` block throws exception A and then auto-close throws exception B, exception A is propagated (it's the primary failure) and exception B is attached to it as a suppressed exception via `A.addSuppressed(B)`. Callers can retrieve suppressed exceptions with `throwable.getSuppressed()`. This preserves the complete failure context. Multiple resources are closed in reverse declaration order, and if multiple close operations fail, all their exceptions are added as suppressed to the original. This is a significant improvement in debuggability for production systems."

PART 5: Concurrency & Multi-Threading

5.1 Thread States and Lifecycle

Definition & Internal Mechanics

A Java thread can be in one of six states defined by `Thread.State`:

State	Description	How to Enter
<code>NEW</code>	Thread created but not started	<code>new Thread(r)</code>
<code>RUNNABLE</code>	Running or ready to run	<code>.start()</code> , <code>unpark()</code> , blocking op returns
<code>BLOCKED</code>	Waiting to acquire a monitor lock	<code>synchronized</code> block while lock is held by another
<code>WAITING</code>	Waiting indefinitely for notification	<code>wait()</code> , <code>join()</code> , <code>LockSupport.park()</code>
<code>TIMED_WAITING</code>	Waiting with a timeout	<code>sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code> , <code>tryLock(ms)</code>
<code>TERMINATED</code>	<code>run()</code> completed	Normal exit or uncaught exception

Key distinctions:

- `BLOCKED` vs `WAITING`: `BLOCKED` threads are waiting for a **monitor lock** (at a `synchronized` entry point). `WAITING` threads have **voluntarily given up** the CPU and wait for explicit notification.
- `BLOCKED` threads remain visible to the thread scheduler as candidates for lock acquisition. `WAITING` threads are descheduled until notified.
- Only `RUNNABLE` threads actually execute on the CPU.

Production-Ready Code Snippet

```
java
```

```

import java.util.concurrent.locks.LockSupport;

/**
 * Demonstrates all six thread states and transitions.
 */
public class ThreadStatesDemo {

    public static void main(String[] args) throws InterruptedException {
        Object lock = new Object();

        // NEW state
        Thread t = new Thread(() -> {
            synchronized (lock) {
                try {
                    lock.wait(2000); // WAITING state
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });
        System.out.println("State after new: " + t.getState()); // NEW

        // Acquire the lock so the thread blocks when it tries
        synchronized (lock) {
            t.start();
            Thread.sleep(50); // Let t reach synchronized(lock)
            System.out.println("State while we hold lock: " + t.getState()); // BLOCKED
        }
        // Now we release the lock, t enters wait()
        Thread.sleep(50);
        System.out.println("State after wait(): " + t.getState()); // WAITING (or TIMED_WAITING)

        Thread sleeping = new Thread(() -> {
            try { Thread.sleep(5000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        sleeping.start();
        Thread.sleep(50);
        System.out.println("Sleeping thread state: " + sleeping.getState()); // TIMED_WAITING
        sleeping.interrupt();
        sleeping.join();
        System.out.println("After interrupt: " + sleeping.getState()); // TERMINATED

        t.join();
    }
}

```

```
        System.out.println("Final state: " + t.getState()); // TERMINATED
    }
}
```

Common Interview Question & Expert Answer

Q: "What is the difference between **BLOCKED** and **WAITING** thread states? Can a **WAITING** thread be notified into **BLOCKED**?"

A: "Yes — and this is a subtlety many candidates miss. A thread in **WAITING** state has called `wait()`, `join()`, or `LockSupport.park()`. It's not competing for any lock. When another thread calls `notify()` or `notifyAll()`, the waiting thread transitions to **BLOCKED** — not directly to **RUNNABLE**. This is because `wait()` releases the monitor lock while waiting, so when notified, the thread must **re-acquire** that lock before returning from `wait()`. If another thread holds the monitor, the notified thread waits in **BLOCKED** state until the lock is available. So the sequence is: **WAITING** (holds no lock) → notified → **BLOCKED** (competing for lock) → lock acquired → **RUNNABLE**. This is why `notifyAll()` can cause a 'thundering herd' — all waiting threads wake up and immediately contend for the same lock, most of them going back to **BLOCKED**."

5.2 Virtual Threads (Java 21 — Project Loom)

Definition & Internal Mechanics

Virtual threads are lightweight threads managed by the JVM, not the OS. They are cheap to create (thousands or millions), have a tiny stack footprint, and are scheduled onto a pool of **carrier threads** (typically equal to the number of CPU cores).

How they work:

- Virtual threads run on **carrier threads** (platform/OS threads)
- When a virtual thread performs a **blocking operation** (I/O, `sleep`, `lock`), the JVM **unmounts** it from the carrier thread — the carrier thread is freed to run another virtual thread
- When the blocking operation completes, the virtual thread is **remounted** on an available carrier thread to continue
- The entire stack frame of the virtual thread is stored in the **heap** (not OS stack), enabling millions of concurrent virtual threads

Pinning: A virtual thread is **pinned** to its carrier when:

- Inside a `synchronized` block or method (the carrier cannot be freed)

- Calling a native method
- Pinning prevents carrier thread sharing and reduces scalability — use `ReentrantLock` instead of `synchronized` in virtual thread code

When to use: High-concurrency I/O-bound workloads (HTTP servers, database queries, file I/O).
NOT beneficial for CPU-bound tasks (no carrier thread freed during computation).

Production-Ready Code Snippet

```
java
```

```

import java.time.Duration;
import java.util.concurrent.*;

/**
 * Demonstrates virtual threads: creation, scaling behavior, and
 * comparison with platform threads for I/O-bound workloads.
 */
public class VirtualThreadsDemo {

    // Simulates an I/O-bound task (e.g., database query)
    static String simulateDbQuery(int requestId) throws InterruptedException {
        Thread.sleep(Duration.ofMillis(100)); // Simulates network I/O
        return "Result-" + requestId;
    }

    public static void main(String[] args) throws Exception {
        int taskCount = 10_000;

        // Virtual threads – can scale to millions
        long start = System.currentTimeMillis();
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> futures = new ArrayList<>(taskCount);
            for (int i = 0; i < taskCount; i++) {
                final int id = i;
                futures.add(executor.submit(() -> simulateDbQuery(id)));
            }
            for (Future<String> f : futures) f.get(); // Wait for all
        }
        long virtualTime = System.currentTimeMillis() - start;

        System.out.printf("Virtual threads: %,d tasks in %d ms%n", taskCount, virtualTime);

        // Creating a single named virtual thread
        Thread vThread = Thread.ofVirtual()
            .name("my-virtual-thread")
            .start(() -> System.out.println("Running on: " +
                Thread.currentThread() + " (virtual=" +
                Thread.currentThread().isVirtual() + ")"));
        vThread.join();

        // Structured Concurrency (Java 21)
        try (StructuredTaskScope.ShutdownOnFailure scope = new StructuredTaskScope.ShutdownOnFailure()) {
            StructuredTaskScope.Subtask<String> task1 = scope.fork(() -> simulateDbQuery(1));
        }
    }
}

```

```

        StructuredTaskScope.Subtask<String> task2 = scope.fork(() -> simulateDbC
scope.join().throwIfFailed(); // Wait for both; throw if any failed
System.out.println("Structured results: " + task1.get() + ", " + task2.g
    }
}
}

```

Common Interview Question & Expert Answer

Q: "Virtual threads are scheduled by the JVM, not the OS. What is 'pinning,' and why does it undermine virtual thread scalability?"

A: "Pinning occurs when a virtual thread cannot be unmounted from its carrier thread during a blocking operation. This happens in two main cases: (1) inside a `synchronized` block or method, and (2) when executing native methods. In these cases, the virtual thread holds the carrier thread hostage — the carrier is blocked and cannot run other virtual threads. If all carrier threads (typically equal to `Runtime.getRuntime().availableProcessors()`) are pinned, the application grinds to a halt despite having many virtual threads waiting. This is why code using virtual threads should prefer `ReentrantLock` (from `java.util.concurrent.locks`) over `synchronized` — `ReentrantLock` supports unmounting, while `synchronized` does not (as of Java 21; future JDK versions aim to fix `synchronized`). You can detect pinning by running with the JVM flag `-Djdk.tracePinnedThreads=full`, which logs whenever a virtual thread is pinned. In practice, for new code, replace `synchronized` with `ReentrantLock` in hot paths. Library code that uses `synchronized` internally (like some database drivers) may still cause pinning — this is an active area of JDK and ecosystem migration work."

5.3 The `volatile` Keyword and Java Memory Model

Definition & Internal Mechanics

The **Java Memory Model (JMM)** defines how threads interact through memory. Without synchronization, threads may cache values in CPU registers or L1/L2 caches and never flush to main memory, causing **visibility problems**.

`volatile` guarantees:

1. **Visibility:** Every write to a `volatile` variable is immediately flushed to main memory. Every read of a `volatile` variable reads from main memory (not a cached copy).
2. **Ordering:** Establishes a **happens-before** relationship: all actions before a `volatile` write are visible to any thread that reads that `volatile` variable after the write.

3. **Prevents instruction reordering:** The JVM/CPU cannot reorder instructions across a `volatile` access (acts as a memory barrier).

`volatile` does NOT guarantee:

- **Atomicity of compound operations.** `volatile int counter; counter++` is NOT atomic — it's a read-modify-write (three separate operations).

When to use `volatile`:

- A flag read by multiple threads and written by one thread (e.g., `volatile boolean running`)
- A reference that needs to be visible across threads after a single write

Production-Ready Code Snippet

```
java
```

```

import java.util.concurrent.atomic.AtomicInteger;

/**
 * Demonstrates volatile visibility, happens-before guarantee,
 * and why volatile alone is insufficient for compound operations.
 */
public class VolatileDemo {

    static volatile boolean running = true; // volatile flag – visible across threads
    static volatile int latestValue = 0;

    // NON-atomic: volatile int cannot safely do counter++
    static volatile int unsafeCounter = 0;
    static AtomicInteger safeCounter = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            int localCount = 0;
            while (running) { // Reads volatile 'running' from main memory each iteration
                localCount++;
            }
            System.out.println("Worker stopped. Local count: " + localCount);
        });
        worker.start();

        Thread.sleep(100); // Let worker run a bit
        running = false; // Write visible to worker immediately due to volatile
        worker.join();
        System.out.println("Worker terminated.");

        // Demonstrating visibility without volatile would be unsafe:
        // Without volatile, the JVM could cache 'running' in a register
        // and the worker thread might loop forever never seeing running=false

        // Demonstrating that volatile != atomic for compound ops
        int threadCount = 100;
        Thread[] threads = new Thread[threadCount];
        for (int i = 0; i < threadCount; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {
                    unsafeCounter++; // NOT atomic: read + increment + write (race condition)
                    safeCounter.incrementAndGet(); // CAS-based atomic operation
                }
            });
        }
    }
}

```

```

    });
    threads[i].start();
}
for (Thread t : threads) t.join();

System.out.println("Expected: 100000");
System.out.println("Unsafe counter (volatile, not atomic): " + unsafeCounter);
System.out.println("Safe counter (AtomicInteger): " + safeCounter.get()); //
}
}

```

Common Interview Question & Expert Answer

Q: "Can you implement a thread-safe counter using only `volatile`? What is the 'happens-before' relationship?"

A: "No. `volatile` only guarantees visibility and ordering — not atomicity of compound operations. `counter++` is three bytecode operations: `GETSTATIC` (read), `IADD` (increment), `PUTSTATIC` (write). Two threads can both read `5`, both increment to `6`, and both write `6` — losing one increment. For a thread-safe counter, you need `AtomicInteger` (using Compare-And-Swap hardware instructions) or a `synchronized` method. The **happens-before** relationship is a formal ordering guarantee in the JMM: 'Action A happens-before action B' means B is guaranteed to see all effects of A. Volatile establishes happens-before: every write to a volatile variable `v` happens-before every subsequent read of `v`. So if Thread 1 writes `x = 10; volatileFlag = true;` and Thread 2 reads `if(volatileFlag) { int y = x; }` — Thread 2 is guaranteed to see `x = 10` because the write to `x` happens-before the volatile write, which happens-before the volatile read, which happens-before the read of `x`. Happens-before is transitive."

5.4 Synchronized Blocks and Lock Inflation

Definition & Internal Mechanics

Every Java object has an associated **monitor** (intrinsic lock). `synchronized` uses this monitor for mutual exclusion. The JVM uses several lock optimization strategies:

Lock States (from cheapest to most expensive):

1. **Unlocked:** No thread holds the lock
2. **Biased Locking (removed in Java 15):** When a lock is acquired by the same thread repeatedly, the JVM biases the lock to that thread — subsequent acquisitions by the same thread require only a read (no CAS). Was removed due to maintenance complexity.

3. **Lightweight Lock:** When a lock is contended by a second thread, it inflates to a lightweight lock using **CAS (Compare-And-Swap)** on the object header. The waiting thread **spin-waits** briefly.
4. **Heavyweight (Inflated) Lock:** If the lightweight CAS fails repeatedly (actual contention), the lock inflates to a heavyweight monitor — the OS mutex is used, and waiting threads are **parked** (descheduled by the OS).

Lock inflation is one-way: A lock that inflates to heavyweight stays heavyweight for the object's lifetime (in practice), because deflation requires careful coordination.

Production-Ready Code Snippet

```
java
```

```

import java.util.concurrent.locks.*;

/**
 * Demonstrates synchronized blocks, object monitors,
 * and ReentrantLock with timed/interruptible acquisition.
 */
public class SynchronizationDemo {

    static class BankAccount {
        private double balance;
        private final String id;
        private final Lock lock = new ReentrantLock(true); // fair: true

        public BankAccount(String id, double initialBalance) {
            this.id = id;
            this.balance = initialBalance;
        }

        // synchronized method – uses 'this' as monitor
        public synchronized void depositSync(double amount) {
            if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive");
            balance += amount;
        }

        // ReentrantLock with timeout – avoids indefinite blocking (deadlock prevention)
        public boolean transferTo(BankAccount target, double amount) {
            try {
                if (this.lock.tryLock(500, java.util.concurrent.TimeUnit.MILLISECONDS)) {
                    try {
                        if (target.lock.tryLock(500, java.util.concurrent.TimeUnit.MILLISECONDS)) {
                            try {
                                if (balance < amount) {
                                    System.out.println("Insufficient funds in " + id);
                                    return false;
                                }
                                this.balance -= amount;
                                target.balance += amount;
                                System.out.printf("Transferred $%.2f from %s to %s\n",
                                    amount, id, target.id);
                                return true;
                            } finally {
                                target.lock.unlock();
                            }
                        }
                    } finally {
                        this.lock.unlock();
                    }
                }
            }
        }
    }
}

```

```

        } else {
            System.out.println("Could not acquire target lock – avoid");
            return false;
        }
    } finally {
        this.lock.unlock();
    }
} else {
    System.out.println("Could not acquire source lock");
    return false;
}
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return false;
}
}

public synchronized double getBalance() { return balance; }
}

public static void main(String[] args) throws InterruptedException {
    BankAccount alice = new BankAccount("Alice", 1000.0);
    BankAccount bob = new BankAccount("Bob", 500.0);

    Thread t1 = new Thread(() -> alice.transferTo(bob, 200.0));
    Thread t2 = new Thread(() -> bob.transferTo(alice, 100.0));
    t1.start(); t2.start();
    t1.join(); t2.join();

    System.out.printf("Alice: $%.2f, Bob: $%.2f\n",
        alice.getBalance(), bob.getBalance());
}
}

```

Common Interview Question & Expert Answer

Q: "What is the difference between `synchronized` and `ReentrantLock`? When would you choose one over the other?"

A: "`ReentrantLock` provides everything `synchronized` does, plus: (1) **Timed lock acquisition** via `tryLock(timeout)` — avoid waiting indefinitely, useful for deadlock prevention. (2) **Interruptible lock acquisition** via `lockInterruptibly()` — a thread waiting for a lock can be interrupted. (3) **Fairness policy** — `new ReentrantLock(true)` grants locks in FIFO order,

preventing starvation. (4) **Multiple condition variables** via `lock.newCondition()` — allows more granular wait/signal than `wait()`/`notify()` which are tied to the entire object monitor. (5) **Non-block-structured release** — you can acquire in one method and release in another (though this requires careful design). I choose `synchronized` for simple cases — it's concise, automatically released (even on exception), and the JVM has optimized it heavily. I choose `ReentrantLock` when I need timed acquisition, interruptibility, fairness guarantees, or multiple condition variables. The critical mistake with `ReentrantLock` is forgetting to `unlock()` in a `finally` block — unlike `synchronized`, it won't auto-unlock if an exception is thrown."